

Continuous Collision Detection Trust Regions for Clothing Simulation

ANONYMOUS AUTHOR(S)

SUBMISSION ID: 1107

We present a trust region technique for simulating clothing with barrier potentials. Our method guarantees that each iteration of a nonlinear Gauss–Seidel solver maintains a collision free state. While previous techniques have achieved this by limiting step sizes based on geometric proximity heuristics, we show that continuous collision detection can be used instead to greatly increase step sizes and, in turn, dramatically decrease computation time. We define our trust regions as axis aligned bounding boxes that surround each vertex. By constraining each Gauss–Seidel vertex correction to remain in the respective box, we establish a worst-case collection of vertex/triangle and edge/edge interaction pairs that can occur over the time step. These interactions occur through barrier potentials and continuous collision detection step size limitation. Using this interaction information (together with elastic mesh connectivity) we color vertices that can be safely updated in parallel. Notably, when vertices in a given color are being simultaneously updated (on different threads), our coloring guarantees that the continuous collision detection check involves only one moving vertex (since the remaining three vertices in the edge/edge or vertex/triangle pair are of different colors). This reduces the notoriously rounding-sensitive cubic solve for potential collision to a simple linear solve. We demonstrate the efficacy of our method in representative clothing simulation scenarios.

ACM Reference Format:

Anonymous Author(s). 2026. Continuous Collision Detection Trust Regions for Clothing Simulation. 1, 1 (May 2026), 6 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 INTRODUCTION

The simulation of thin elastic materials in close contact is a ever present in physics based animation. For example, virtual characters often wear a variety of layered garments (usually in the form of triangle meshes) that must collide with each other to prevent visual artifacts. While elastic constitutive forces in the cloth naturally use the static structure of the mesh, collision forces utilize different portions of the mesh at different times and are generally much more challenging to resolve [1]. Furthermore, the continuous collision detection (CCD) problem where each point/triangle and segment/segment mesh pair is prevented from colliding over the course of each time step is also required to guarantee a collision free state [2]. In this problem, mesh vertices are assumed to follow linear trajectories over a time step, which reduces the determination of if and when a point/triangle or segment/segment pair will collide to examining their proximity at the roots of a cubic that lie in the time step interval. This problem is both costly and sensitive to rounding errors [3, 23]. Despite these complications, cloth simulators must

navigate these challenges when collision-free guarantees are required.

Simulation techniques use a wide variety of models for the collision forces and for solving the CCD problem. The most common collision forces are based on simple quadratic potential energy springs, but they are limited in their ability to prevent all collision [7]. Incremental potential contact (IPC) techniques have shown that a singular potential can be used to better prevent collision [18]. However, these require the addition of a CCD-informed line search to backtrack and prevent crossing the singularity. Li et al. [18] create a hypothetical step direction using Newton’s method and then examine the first time that a collision would occur between point/triangle and/or segment/segment pairs. The step size is then uniformly truncated across all vertices based on this worst-case collision time. Impulse-based techniques avoid issues associated with singular barrier potentials and instead iteratively resolve collision pairs by adding momentum conserving impulses one pair at a time [2]. However, because each impulsive correction only considers one pair at a time, it can cause collision in neighboring pairs. This is addressed by iterating over all pairs over and over, but it is not guaranteed to converge to a collision free state in practice. Indeed large iteration counts for these techniques can lead to excessive run times.

Recently, trust region techniques have been adopted that circumvent the need for the CCD check [5, 25]. In these approaches, each mesh vertex computes a safe radius that it is allowed to move within. By conservatively examining the proximity between each point/triangle and edge/edge pair in the mesh (and assuming a collision free state), a safe step size can be determined for each vertex (e.g. less than half the minimum distance between the point/triangle and edge/edge pairs it is in). Chen et al. [5] use this idea is used with a barrier potential. They use the Vertex Block Descent (VBD) Gauss–Seidel technique [6] and allow each vertex to move independently (within its trust region). This avoids the problem of Li et al. [18] where step size is based on a distant/worst case colliding pair. However, the safe radius can be overly small and convergence limiting in practice (e.g. consider regions of the mesh in close proximity, but with separating velocities).

While the Gauss–Seidel nature of VBD allows for localized collision step limiting, a coloring technique is needed for efficient parallel implementation [6, 7]. A greedy coloring of vertices based on elastic mesh and collision interactions is needed so that vertices in a given color do not interact (via elastic or collision forces). However, this is costly with the introduction of the barrier potential. As a vertex changes position during the Gauss–Seidel iteration, it will potentially cause barrier interactions between different point/triangle and edge/edge pairs. When this happens, the coloring information needs to be updated, which is costly given the frequency with which this

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM XXXX-XXXX/2026/5-ART
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

can occur. Chen et al. [5, 6] avoid re-coloring based on new barrier interactions, however this degrades the iteration (from Gauss–Seidel) to the slower convergencing Jacobi iteration. Chen et al. [7] preserve the Gauss–Seidel behavior but avoid this problem by using a simple quadratic spring barrier potential that changes once per time step (and therefore loses all collision-free guarantees).

We design a trust region technique that utilizes CCD to allow for larger step sizes and a natural vertex coloring that can be done efficiently. Like Chen et al. [5], we use a barrier potential and Gauss–Seidel iteration to avoid the global step size limitation in Li et al. [18]. However, by using CCD we remove the overly conservative per-vertex step size restrictions of Wu et al. and Chen et al. [5, 25]. To achieve this, we restrict vertices to move inside axis-aligned bounding boxes (AABBs) during the iterative update. By restricting vertex motion in this way, edges and triangles move in predictable boxes created by their vertices. We can then determine worst-case point/triangle and edge/edge interactions so that every vertex knows a super-set of collision pairs that its motion may effect over the update. Because our barrier energy is also based on these pairs we then also know a super set of which pairs will effect the barrier potential (and therefore subsequent collision force interactions). This gives us enough interaction information to create a vertex coloring that does not need to be redone until the vertex AABBs are allowed to change (subject to user control). The added benefit of this coloring is that vertices in the same color do not interact through CCD, barrier potentials or elastic forces. This means that point/triangle and edge/edge CCD checks associated with the a vertex update are done with only one moving vertex. This reduces the rounding-sensitive cubic solve for collision times to a simple linear solve, dramatically reducing run-time, code complexity and rounding sensitivity. We demonstrate that our CCD trust region technique guarantees a collision free state and dramatically improves run-time while allowing for larger step sizes in collision intensive scenarios.

2 RELATED WORK

Incremental Potential Contact. Li et al. [18] introduced Incremental Potential Contact (IPC), a variational framework that guarantees intersection- and inversion-free dynamics for deformable bodies through log-barrier energies and continuous collision detection. Ferguson et al. [10] extended IPC to rigid body dynamics. Ferguson et al. [9] further extended IPC to high-order finite elements on curved meshes, mapping contact forces between a linear collision proxy and the high-order representation. Li et al. [19] generalized the framework to codimensional structures including shells, rods, and their coupling. Lan et al. [16] introduced affine body dynamics, which relaxes rigidity to an affine motion model for orders-of-magnitude speedups on stiff materials while preserving IPC’s non-intersection guarantees. Huang et al. [14] proposed GIPC, a GPU-optimized IPC solver that derives analytic eigensystems for the barrier Hessian, avoiding numerical eigendecomposition and enabling significant acceleration. Huang et al. [15] further introduced StiffGIPC, combining a connectivity-enhanced multilevel preconditioner with cubic strain-limiting energies for stiff membranes such as cloth, achieving up to 10× speedup over prior GPU IPC methods. Li et al. [20]

combined subspace preconditioning with GPU Jacobi iteration for cloth simulation, coupling IPC contact handling with projective dynamics.

Collision detection and avoidance. Bridson et al. [2] established the standard continuous collision detection framework based on coplanarity tests that we adopt for step filtering. Brochu et al. [4] showed that the underlying cubic polynomial solver is vulnerable to rounding error, particularly in near-planar configurations. Ericson [8] provides a comprehensive reference for bounding volume hierarchies and swept AABB queries. Wu et al. [25] proposed a condition-based collision avoidance method for cloth self-collisions, avoiding explicit CCD. Wang et al. [24] introduced a large-scale CCD benchmark and proposed the Tight-Inclusion algorithm, an inclusion-based method that provides conservative, gap-free collision detection with support for minimum separation. Chen et al. [5] introduced offset geometric contact, which enforces separation constraints via geometric offsets and trust-region step filtering as an alternative to CCD-based line search.

Optimization time integration. Gast et al. [12] formulated implicit time integration as an optimization problem, enabling large stable time steps for elasticity simulation. The incremental potential formulation [18] builds on this variational perspective, and our solver inherits the same structure while replacing the global minimization with coordinate-wise descent.

Position-based methods. Macklin et al. [21] introduced XPBD, extending position-based dynamics to handle compliant constraints with implicit time stepping. While efficient and widely adopted, XPBD solves a compliance-regularized system whose solutions depend on iteration count and do not converge to a physically consistent equilibrium.

Nonlinear Gauss–Seidel methods. Fratarcangeli et al. [11] introduced a parallel Gauss–Seidel method using randomized graph coloring for soft body dynamics. Chen et al. [6] proposed vertex block descent (VBD), a vertex-level Gauss–Seidel method for the variational implicit Euler problem that achieves unconditional stability and convergent solutions with maximized parallelism. Chen et al. [7] applied a similar nonlinear Gauss–Seidel strategy to quasistatic hyperelasticity, achieving fast convergence without global linear solves. Lan et al. [17] proposed a GPU Jacobi/Gauss–Seidel method that mitigates overshoot to achieve near second-order convergence comparable to Newton’s method. Our method extends these ideas to the dynamic setting with contact, incorporating IPC barrier energies, continuous collision detection, and a parallel batching scheme based on conflict graph coloring.

Constitutive models. We use the corotated elastic energy of Stomakhin et al. [22] for membrane stretching and the discrete shell bending model of Grinspun et al. [13]. Both are standard choices for thin shell simulation.

3 OVERVIEW

We represent our thin elastic materials with triangle meshes. We use $\mathbf{x}^n \in \mathbb{R}^{3N_V}$ to denote the array of mesh vertex positions at time t^n (where N_V is the number of vertices in the mesh). $\mathbf{v}^n \in \mathbb{R}^{3N_V}$

represents the array of mesh vertex velocities. We use the notation $\mathbf{x}_i^n \in \mathbb{R}^3$ and $\mathbf{v}_i^n \in \mathbb{R}^3$ to represent the components corresponding to vertex $1 \leq i \leq N_V$. Similarly we use m_i to denote the mass of the vertex. Mesh edges $e = (v_a, v_b)$ and triangles $t = (v_a, v_b, v_c)$ are monitored for collision proximity. Here $1 \leq v_a, v_b, v_c \leq N_V$ refer to the vertices in the edges and triangles. Lastly, we use $V = \{1, 2, \dots, N_V\}$ to denote the collection of all vertex indices.

Our method is designed to solve the problem

$$\mathbf{x}^{n+1} = \arg \min_{\mathbf{y} \in \mathbb{R}^{3N_V}} \Phi(\mathbf{y}), \quad \mathbf{v}^{n+1} = \frac{\mathbf{x}^{n+1} - \mathbf{x}^n}{\Delta t} \quad (1)$$

where $\Phi(\mathbf{y}) = \frac{1}{2} \|\mathbf{x} - \hat{\mathbf{x}}\|_{\mathbf{M}}^2 + (\Delta t)^2 PE(\mathbf{x})$ is the incremental potential [10, 18] and $\hat{\mathbf{x}} = \mathbf{x}^n + \Delta t \mathbf{v}^n$ is the inertial extrapolation with $\|\mathbf{v}\|_{\mathbf{M}}^2 = \mathbf{v}^T \mathbf{M} \mathbf{v}$ and $\mathbf{M} = \text{diag}(m_1 \mathbf{I}_3, \dots, m_{N_V} \mathbf{I}_3)$. The potential energy is a sum of the corotated elastic energy, bending energy, gravitational potential, collision barrier potential and pinning penalty respectively $PE(\mathbf{y}) = PE^e(\mathbf{y}) + PE^b(\mathbf{y}) + PE^g(\mathbf{y}) + PE^{\text{bar}}(\mathbf{y}) + PE^{\text{pin}}(\mathbf{y})$. We refer the reader to the supplementary technical document for more detail about these potentials, however we note that the barrier potential $PE^{\text{bar}}(\mathbf{y})$ is only active when point/triangle and edge/edge pairs are within a characteristic distance $\hat{d}$. We solve this problem using nonlinear Gauss-Seidel [6, 7]. We use $\mathbf{g}_i = \frac{\partial \Phi}{\partial \mathbf{y}_i} \in \mathbb{R}^3$ and $\mathbf{H}_i = \frac{\partial^2 \Phi}{\partial \mathbf{y}_i^2} \in \mathbb{R}^{3 \times 3}$ to denote the vertex-wise gradient and Hessian respectively. With this convention, the Gauss-Seidel update is simply

$$\mathbf{x}_i^{n+1} \leftarrow \mathbf{x}_i^{n+1} - s_i \mathbf{H}_i^{-1} \mathbf{g}_i \quad (2)$$

where s_i is a step size chosen from our CCD trust region technique (see Sections 4 and 4.1).

4 TRUST REGIONS

The step size s_i is chosen so that the update in Equation 2 does not cause the vertex to leave an AABB and so that it does not violate a point/triangle or edge/edge CCD check (see Section 4.1). For each node i , we construct an isotropic (blue) box B_i the axis-aligned cube centered at a position $\mathbf{x}_i$ with half-width r_i . The center of the box $\mathbf{x}_i$ is updated after a user-specified number of Gauss-Seidel iterations. This is typically done just once or twice per time step for efficiency. We choose the half-width adaptively based on displacement history

$$r_i = \text{clamp}(\eta \|\Delta \mathbf{x}_i^{\text{prev}}\|, r_{\min}, r_{\max}), \quad (3)$$

where $\Delta \mathbf{x}_i^{\text{prev}}$ is the displacement of node i during the previous time step and $\eta = 1.2$ is temporal padding factor. This heuristic adapts the box sizes to local dynamics: nodes that moved little in the previous time step receive small boxes, tightening the broad phase and reducing the number of candidate pairs, while nodes undergoing large displacements receive correspondingly larger boxes.

For each segment $e = (v_a, v_b)$ and triangle $t = (v_a, v_b, v_c)$, a red box is formed as the axis-aligned bounding box of the convex hull of its member blue boxes

$$R_e = \text{conv}(B_{v_a} \cup B_{v_b}), \quad R_t = \text{conv}(B_{v_a} \cup B_{v_b} \cup B_{v_c}). \quad (4)$$

Each red box is then inflated by the characteristic length of the barrier potential $\hat{d}$ to produce a green box

$$G_e = R_e \oplus \hat{d}, \quad G_t = R_t \oplus \hat{d}, \quad (5)$$

where $\oplus$ denotes Minkowski padding. Since every node is confined to its blue box, each red box encloses the swept volume of its primitive, and each green box conservatively encloses the $\hat{d}$ -neighborhood of that swept volume.

4.1 Pair Registration and CCD

A point-triangle pair (n, t) is registered whenever $B_n \cap G_t \neq \emptyset$ and n is not a vertex of t . Edge-edge pairs (e_1, e_2) are registered whenever $R_{e_1} \cap G_{e_2} \neq \emptyset$ and the two edges share no vertex. These intersection tests are accelerated by constructing bounding volume hierarchies over the green triangle and green edge boxes, respectively. Pair registration is performed whenever vertex (blue) boxes are updated. Because each blue box encloses all positions its node can reach during the update in Equation (2), and each green box conservatively encloses the $\hat{d}$ -neighborhood of its primitive's reachable volume, every primitive pair that could come within distance $\hat{d}$ during the Gauss-Seidel sweep is guaranteed to be registered. Therefore, these registered pairs form a super set of all point/triangle and edge/edge pairs that will contribute to the barrier potential during the non-linear Gauss-Seidel sweep obeying the trust regions. Furthermore, these pairs contain any possible edge/edge and point/triangle swept volumes during the update. This means that as long as the initial guess for the Gauss-Seidel iteration was collision free, any subsequent update using Equation (2) will remain collision free as long as the vertex limits s_i based on a CCD check of all registered edge/edge and point/triangle pairs that contain the vertex. Remarkably, since only one node is effected by the update in Equation (2) these CCD checks only require the solution of a simple linear equation. In contrast, the more standard CCD check where all four nodes are moving requires the solution of a costly, rounding sensitive, cubic equation. We outline the entire process in Algorithm 1.

Algorithm 1 Box construction

- 1: **for** each node i **do**
 - 2: $r_i \leftarrow \text{clamp}(\eta \|\Delta \mathbf{x}_i^{\text{prev}}\|, r_{\min}, r_{\max})$
 - 3: **Blue box:** $B_i \leftarrow \text{Box}(\mathbf{x}_i, r_i)$
 - 4: **end for**
 - 5: **Red boxes:** $R_e \leftarrow \text{conv}(\bigcup_{v \in e} B_v)$, $R_t \leftarrow \bigcup_{v \in t} B_v$
 - 6: **Green boxes:** $G_e \leftarrow R_e \oplus \hat{d}$, $G_t \leftarrow R_t \oplus \hat{d}$
 - 7: Register pairs: (n, t) if $B_n \cap G_t \neq \emptyset$; (e_1, e_2) if $R_{e_1} \cap G_{e_2} \neq \emptyset$
-

5 PARALLEL GAUSS-SEIDEL SOLVER

The Gauss-Seidel updates must be done in parallel to maximize efficiency. Two vertices i and j cannot be updated simultaneously if their local subproblems are coupled. We build an undirected conflict graph $G = (V, E)$ with $(i, j) \in E$ whenever i and j belong to the same elastic stencil, or a registered edge/edge or point/triangle pair involves both i and j . The first condition captures coupling from the corotated elastic forces (triangle stencils) and the bending forces (hinge stencils consisting of the four vertices of each pair of

adjacent triangles); the second captures couplings from the collision barrier forces and CCD checks. We emphasize that restricting each vertex update to its blue AABB trust region (and registering pairs accordingly) is the key guaranty that all Gauss–Seidel coupling between nodes i and j are accounted for. We partition V into independent color sets $C_1, \dots, C_K$ by greedy graph coloring of $G = (V, E)$. Within each color, no two nodes share a mesh triangle or a registered edge/edge or point/triangle pair. Therefore every node in a given color can be updated via Equation (2) at the same time without race conditions. Furthermore, this guarantees that the CCD step size limitation (Section 4.1) is still done with only one node moving/no cubic solve. Algorithm 2 summarizes the complete parallel Gauss–Seidel time step procedure.

Algorithm 2 Parallel CCD Trust Region Gauss–Seidel

```

1: Save positions:  $\mathbf{x}_i^{\text{start}} \leftarrow \mathbf{x}_i^{n+1}$ 
2: for  $\ell = 1, \dots, N_{\text{max}}$  do
3:   if  $\text{mod}(\ell, \text{rebuild\_count}) = 0$  then
4:     Rebuild boxes (Algorithm 1)
5:     Build conflict graph and color
6:   end if  $C_1, \dots, C_K$ 
7:   for each color  $k = 1, \dots, K$  do
8:     for each node  $i \in C_k$  (in parallel) do
9:       Compute  $\mathbf{H}_{ii} \delta_i = -\mathbf{g}_i$ 
10:      Clamp  $s_i$  so that  $\mathbf{x}_i^{n+1} + s_i \delta_i$  is in blue box  $B_i$ 
11:      Clamp  $s_i$  based on single-node linear CCD step (based
        on transition from  $\mathbf{x}_i^{n+1}$  to  $\mathbf{x}_i^{n+1} + s_i \delta_i$ )
12:    end for
13:  end for
14: end for
15: Update displacement history for blue box sizing:  $\Delta \mathbf{x}_i^{\text{prev}} \leftarrow$ 
     $\mathbf{x}_i^{n+1} - \mathbf{x}_i^{\text{start}}$ .
```

6 NUMERICAL EXPERIMENT

We evaluate our method on three cloth simulation scenarios of increasing complexity, exercising self-collision, cloth–obstacle contact, and multi-layer interaction. All simulations run at 30 frames per second. Material and solver parameters are listed in Table 1; runtime statistics are reported in Table 2. All experiments were performed on a server with an AMD EPYC 9554P 64-core processor (3.76 GHz max boost) and 377 GB RAM, running Ubuntu 22.04. We used 64 threads for all benchmarks.

Twisting cloth. A square cloth sheet is clamped along two opposite edges and the clamps are spun in opposite directions about the shared midline. The relative twist accumulates at 0.5 turns per second, reaching four full turns over the reference run. As the twist tightens, the sheet develops deep helical folds that bring distant regions of the surface into close proximity, producing tight self-contact and buckling.

Two-cylinder twist. Four closed-loop cloth strips are draped over two parallel horizontal cylinders. The cylinders counter-rotate about their shared axis direction at 0.15 turns/s each, accumulating two

full turns per cylinder before reversing back to the rest configuration. The untwist phase serves as a reversibility test, requiring the solver to cleanly separate surfaces that were in deep contact.

Cylinder wrap. A rectangular cloth strip is draped under a horizontal cylinder with both top edges fixed as static bars on either side. The cylinder yaws about the vertical axis at 0.30 turns/s, twisting the cloth between the fixed clamps and the rotating bottom-wrap region. After reaching 4 full turns, the rotation reverses and the cloth unwinds back to its rest configuration. This scene introduces cloth–obstacle interaction through a signed-distance penalty while simultaneously generating layered self-contact as the twist tightens. The reversal tests whether the solver can cleanly peel apart layers that were pressed together under sustained load.

Table 1. Simulation parameters.

Parameter	Twist	Two-cyl.	Cyl. wrap
Vertices	10 000	12 308	10 136
Triangles	19 602	23 040	19 800
Young’s modulus E (Pa)	115	115	115
Poisson’s ratio ν	0.25	0.25	0.25
Density ρ (kg/m ³)	900	900	900
Thickness h (m)	0.001	0.001	0.001
Bending stiffness k_B (N/m)	0.009	0.009	0.009
Pin stiffness k_{pin} (N/m)	10^8	5×10^6	10^8
Barrier distance $\hat{d}$ (m)	0.005	0.005	0.005
Barrier stiffness κ_{bar} (N/m)	100	100	100
SDF stiffness k_{sdf} (N/m)	—	10^5	10^9
SDF range ϵ_{sdf} (m)	—	0.002	0.002
Substeps / frame	5	3	5
GS sweeps N_{max}	10	10	10
Frames	240	900	850

Table 2. Performance comparison.

	Twist	Two-cyl.	Cyl. wrap
OGC solver	19.82	—	—
Ours + OGC-style steps (s/frame)	4.61	5.23	6.14
Ours (s/frame)	1.90	2.71	2.52

For each scene, the OGC solver requires more substeps per frame (usually around $3\times$ more or less) to achieve comparable visual quality. This is because its trust-region step filtering is more conservative than our CCD-based filtering. We increase the OGC substep count until its results are visually indistinguishable from ours. The reported timings therefore reflect the true computational cost of each method at matched output quality.

REFERENCES

- [1] David Baraff and Andrew Witkin. 1998. Large Steps in Cloth Simulation. In *Proceedings of the 25th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '98)*. ACM Press, 43–54. <https://doi.org/10.1145/280814.280821>
- [2] Robert Bridson, Ronald Fedkiw, and John Anderson. 2002. Robust Treatment of Collisions, Contact and Friction for Cloth Animation. *ACM Transactions on Graphics* 21, 3 (2002), 594–603. <https://doi.org/10.1145/566654.566623>
- [3] Tyson Brochu, Essex Edwards, and Robert Bridson. 2012. Efficient geometrically exact continuous collision detection. *ACM Trans. Graph.* 31, 4, Article 96 (July 2012), 7 pages.
- [4] Tyson Brochu, Essex Edwards, and Robert Bridson. 2012. Efficient Geometrically Exact Continuous Collision Detection. *ACM Transactions on Graphics* 31, 4 (2012), 96:1–96:7. <https://doi.org/10.1145/2185520.2185592>
- [5] Anka He Chen, Jerry Hsu, Ziheng Liu, Miles Macklin, Yin Yang, and Cem Yuksel. 2025. Offset Geometric Contact. *ACM Transactions on Graphics* 44, 4 (2025), 160:1–160:21. <https://doi.org/10.1145/3731205>
- [6] Anka He Chen, Ziheng Liu, Yin Yang, and Cem Yuksel. 2024. Vertex Block Descent. *ACM Transactions on Graphics* 43, 4 (2024), 116:1–116:16. <https://doi.org/10.1145/3658179>
- [7] Yizhou Chen, Yushan Han, Jingyu Chen, Zhan Zhang, Alex McAdams, and Joseph Teran. 2024. Position-Based Nonlinear Gauss-Seidel for Quasistatic Hyperelasticity. *ACM Transactions on Graphics* 43, 4 (2024), 115:1–115:15. <https://doi.org/10.1145/3658154>
- [8] Christer Ericson. 2004. *Real-Time Collision Detection*. Morgan Kaufmann, San Francisco, CA.
- [9] Zachary Ferguson, Pranav Jain, Denis Zorin, Teseo Schneider, and Daniele Panozzo. 2023. High-Order Incremental Potential Contact for Elastodynamic Simulation on Curved Meshes. In *ACM SIGGRAPH 2023 Conference Proceedings*. Association for Computing Machinery, Los Angeles, CA, USA, 77:1–77:11. <https://doi.org/10.1145/3588432.3591488>
- [10] Zachary Ferguson, Minchen Li, Teseo Schneider, Francisca Gil-Ureta, Timothy Langlois, Chenfanfu Jiang, Denis Zorin, Danny M. Kaufman, and Daniele Panozzo. 2021. Intersection-Free Rigid Body Dynamics. *ACM Transactions on Graphics* 40, 4 (2021), 1–16. <https://doi.org/10.1145/3450626.3459802>
- [11] Marco Fratarcangeli, Valentina Tibaldo, and Fabio Pellacini. 2016. Vivace: A Practical Gauss-Seidel Method for Stable Soft Body Dynamics. *ACM Transactions on Graphics* 35, 6 (2016), 214:1–214:9. <https://doi.org/10.1145/2980179.2982437>
- [12] Theodore F. Gast, Craig Schroeder, Alexey Stomakhin, Chenfanfu Jiang, and Joseph M. Teran. 2015. Optimization Integrator for Large Time Steps. *IEEE Transactions on Visualization and Computer Graphics* 21, 10 (2015), 1103–1115. <https://doi.org/10.1109/TVCG.2015.2459687>
- [13] Eitan Grinspun, Anil N. Hirani, Mathieu Desbrun, and Peter Schröder. 2003. Discrete Shells. In *Proceedings of the ACM SIGGRAPH/Eurographics Symposium on Computer Animation (SCA '03)*. Eurographics Association, San Diego, CA, 62–67.
- [14] Kemeng Huang, Floyd M. Chitalu, Huancheng Lin, and Taku Komura. 2024. GIPC: Fast and Stable Gauss-Newton Optimization of IPC Barrier Energy. *ACM Transactions on Graphics* 43, 2 (2024), 23:1–23:18. <https://doi.org/10.1145/3643028>
- [15] Kemeng Huang, Xinyu Lu, Huancheng Lin, Taku Komura, and Minchen Li. 2025. StiffGIPC: Advancing GPU IPC for Stiff Affine-Deformable Simulation. *ACM Transactions on Graphics* 44, 3 (2025), 31:1–31:20. <https://doi.org/10.1145/3735126>
- [16] Lei Lan, Danny M. Kaufman, Minchen Li, Chenfanfu Jiang, and Yin Yang. 2022. Affine Body Dynamics: Fast, Stable and Intersection-Free Simulation of Stiff Materials. *ACM Transactions on Graphics* 41, 4 (2022), 67:1–67:14. <https://doi.org/10.1145/3528223.3530064>
- [17] Lei Lan, Zixuan Lu, Chun Yuan, Weiwei Xu, Hao Su, Huamin Wang, Chenfanfu Jiang, and Yin Yang. 2025. JGS2: Near Second-order Converging Jacobi/Gauss-Seidel for GPU Elastodynamics. *ACM Transactions on Graphics* 44, 4 (2025), 44:1–44:15. <https://doi.org/10.1145/3731183>
- [18] Minchen Li, Zachary Ferguson, Teseo Schneider, Timothy Langlois, Denis Zorin, Daniele Panozzo, Chenfanfu Jiang, and Danny M. Kaufman. 2020. Incremental Potential Contact: Intersection- and Inversion-Free, Large-Deformation Dynamics. *ACM Transactions on Graphics* 39, 4 (2020), 49:1–49:20. <https://doi.org/10.1145/3386569.3392425>
- [19] Minchen Li, Danny M. Kaufman, and Chenfanfu Jiang. 2021. Codimensional Incremental Potential Contact. *ACM Transactions on Graphics* 40, 4 (2021), 170:1–170:24. <https://doi.org/10.1145/3450626.3459767>
- [20] Xuan Li, Yu Fang, Lei Lan, Huamin Wang, Yin Yang, Minchen Li, and Chenfanfu Jiang. 2023. Subspace-Preconditioned GPU Projective Dynamics with Contact for Cloth Simulation. In *SIGGRAPH Asia 2023 Conference Papers*. Association for Computing Machinery, Sydney, NSW, Australia, 1:1–1:12. <https://doi.org/10.1145/3610548.3618157>
- [21] Miles Macklin, Matthias Müller, and Nuttapong Chentanez. 2016. XPBD: Position-Based Simulation of Compliant Constrained Dynamics. In *Proceedings of the 9th International Conference on Motion in Games (MIG '16)*. Association for Computing Machinery, Burlingame, CA, 49–54. <https://doi.org/10.1145/2994258.2994272>
- [22] Alexey Stomakhin, Russell Howes, Craig Schroeder, and Joseph M. Teran. 2012. Energetically Consistent Invertible Elasticity. In *Proceedings of the ACM SIGGRAPH/Eurographics Symposium on Computer Animation (SCA '12)*. Eurographics Association, Lausanne, Switzerland, 25–32.
- [23] Bolun Wang, Zachary Ferguson, Xin Jiang, Marco Attene, Daniele Panozzo, and Teseo Schneider. 2022. Fast and Exact Root Parity for Continuous Collision Detection. *Computer Graphics Forum (Proceedings of Eurographics)* 41, 2 (2022), 9 pages.
- [24] Bolun Wang, Zachary Ferguson, Teseo Schneider, Xin Jiang, Marco Attene, and Daniele Panozzo. 2021. A Large Scale Benchmark and an Inclusion-Based Algorithm for Continuous Collision Detection. *ACM Transactions on Graphics* 40, 5, Article 188 (Oct. 2021), 16 pages.
- [25] Longhua Wu, Botao Wu, Yin Yang, and Huamin Wang. 2020. A Safe and Fast Repulsion Method for GPU-based Cloth Self Collisions. *ACM Transactions on Graphics* 40, 1 (2020), 5:1–5:18. <https://doi.org/10.1145/3430025>

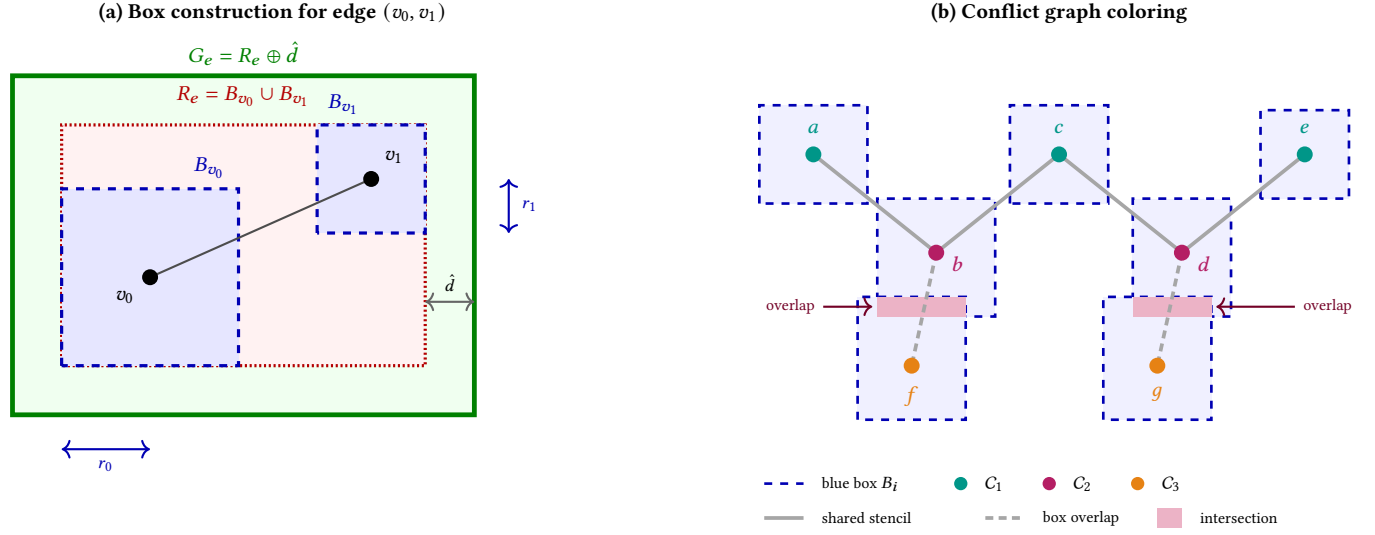


Fig. 1. **(a)** Box construction for edge $e = (v_0, v_1)$. Each node receives an isotropic blue box B_{v_i} with half-width r_i . The red box R_e is the axis-aligned bounding box of $B_{v_0} \cup B_{v_1}$. The green box $G_e = R_e \oplus \hat{d}$ adds barrier-distance padding. **(b)** Conflict graph coloring. Dashed boxes are blue boxes B_i ; node colors indicate batch assignment. Nodes a – e form a mesh chain; f and g belong to a nearby region. Solid gray edges denote conflicts from shared stencils. Dashed gray edges denote conflicts from overlapping blue boxes (purple shading). Three colors suffice: f and g cannot join C_2 because their blue boxes overlap those of b and d , respectively.

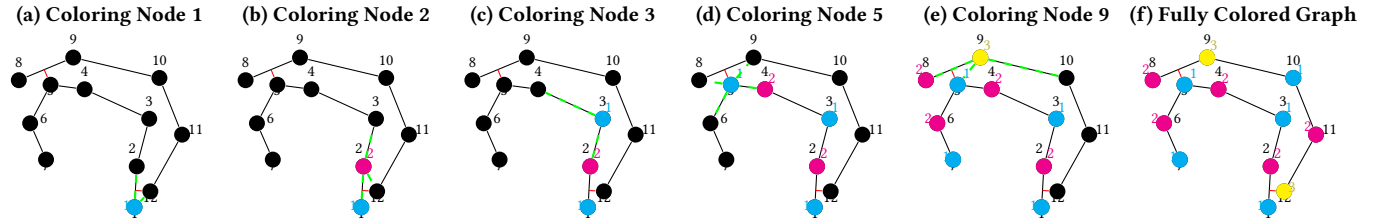


Fig. 2. TODO: description of graph features (a) description of a (b) (c) (d) (e) (f) description of f